



Security Audit

Ithaca (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Behaviours	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	22
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Ithaca team partnered with Hashlock to conduct a security audit of their smart contracts and backend scripts. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

The Ithaca Protocol is a decentralized finance (DeFi) platform that aims to enhance liquidity provision and decentralized governance within the DeFi ecosystem. It employs advanced automated market making (AMM) mechanisms to facilitate seamless trading and liquidity management while incorporating a governance model that empowers stakeholders in decision-making processes. The protocol focuses on efficient liquidity pool management, attractive incentives for liquidity providers, and robust security measures to ensure the protection of user funds. Continuously evolving through its development roadmap, Ithaca Protocol integrates with other DeFi projects to expand its ecosystem and drive sustainable growth.

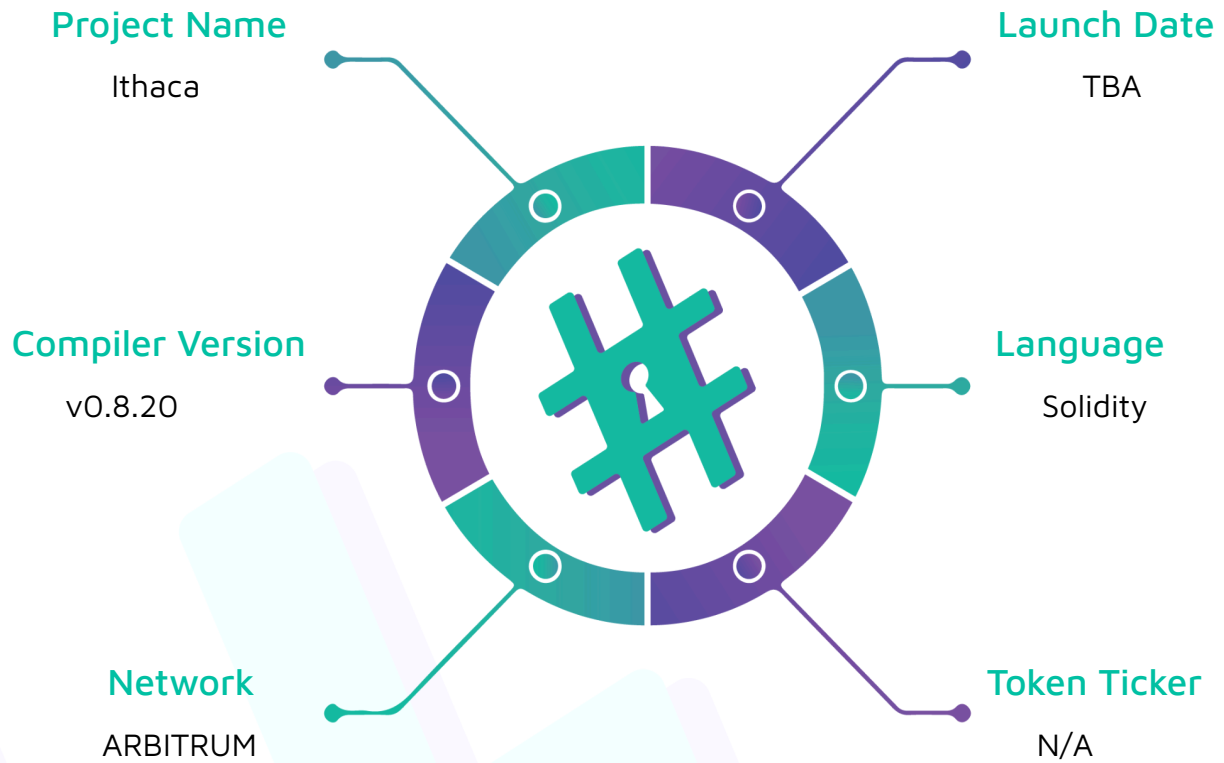
Project Name: Ithaca

Compiler Version: 0.8.20

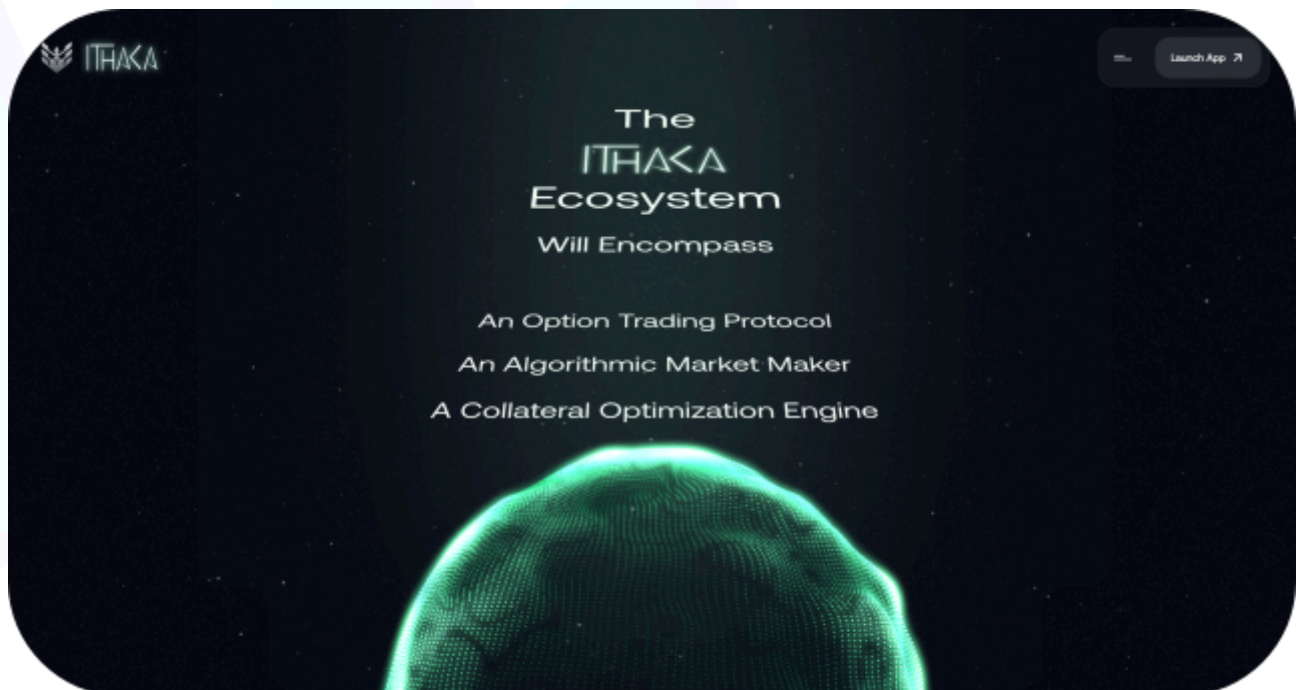
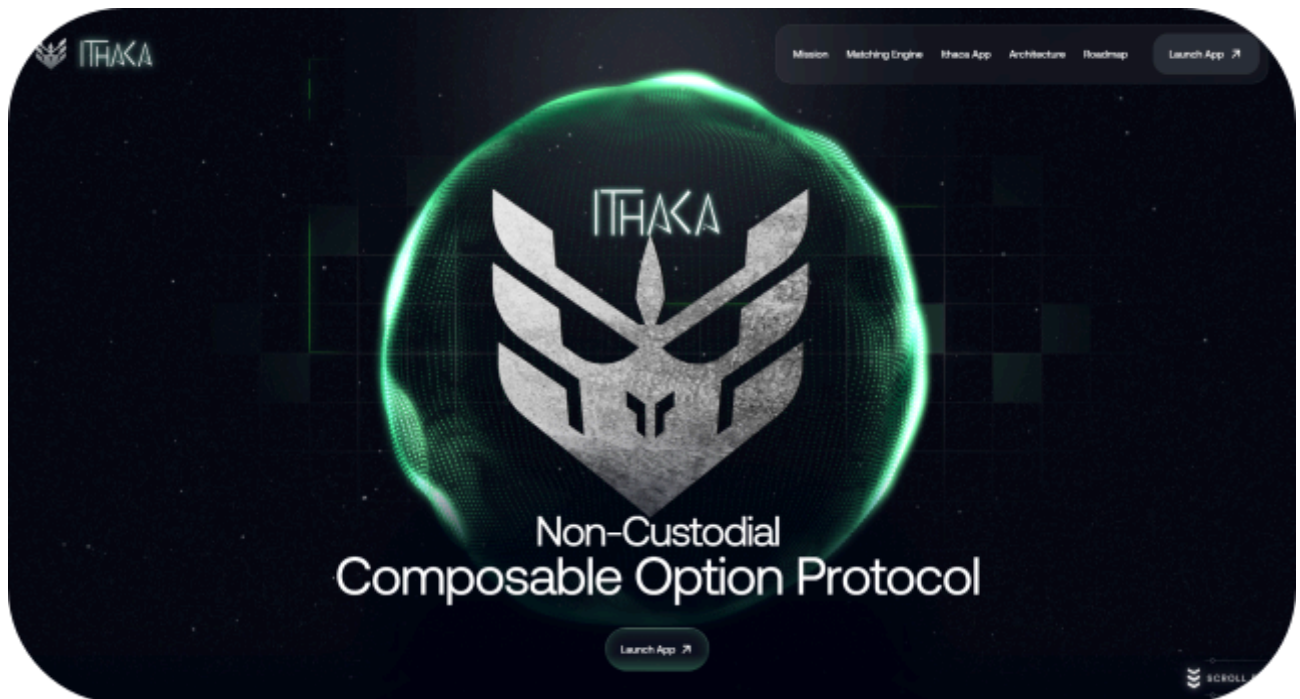
Website: <https://www.ithacaprotocol.io/>

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Ithaca project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Ithaca Protocol Smart Contracts
Platform	Arbitrum / Solidity
Audit Date	September, 2024
Repo 1	https://github.com/ithaca-protocol/protocol-v2/pull/3
Repo 2	https://github.com/ithaca-protocol/ithaca-sc/pull/43
File 1	IthacaFeed.sol
File 1 MD5 Hash	58165008e886236413786ba8fed134d8
File 2	MarginLoanService.java
File 2 MD5 Hash	521d96ac53b7b87836afe08cb85a5f63
File 3	LiquidationHandler.java
File 5 MD5 Hash	5311b4ab312971bbf0a46b0c79be2e0a
File 4	ClientData.java
File 4 MD5 Hash	0374ba3c1202fbaa31a4b4c386019d8e

Security Rating

After Hashlock's audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

We initially identified some significant vulnerabilities that have since been acknowledged.

Hashlock found:

2 High-severity vulnerabilities

2 Medium-severity vulnerabilities

2 Low-severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
IthacaFeed.sol - A contract that retrieves client data or updates it. The update can only be performed by an authorised actor.	Contract achieves this functionality.
MarginLoanService.java - A Java class run periodically used to monitor and manage user accounts. Performs operations such as collect data, process it and perform liquidations if needed.	File achieves this functionality.
LiquidationHandler.java - Used by MarginLoanService to create unwind orders and process liquidations.	File achieves this functionality.
ClientData.java - Used by both MarginLoanService and LiquidationHandler to manage client-specific data and perform operations on client accounts.	File achieves this functionality.

Code Quality

This audit scope involves the smart contracts for the Ithaca project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Ithaca project smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] MarginLoanService - Periodic approach to liquidation running may lead to bad debt in times of volatility

Description

Liquidations are very sensitive to market volatility. The liquidation process runs on a fixed schedule, defined by `marginloan.period.ms` in the configuration.

Each run of the process is treated as a "cycle". There's no evident mechanism for triggering liquidations outside of these periodic cycles.

In times of high volatility, they might need to be run immediately on health check drops. A periodic run approach to liquidations might not be enough to rescue all positions. In such cases, they might become insolvent. The system lacks a mechanism for real-time or event-driven liquidation triggers, which could result in positions becoming severely undercollateralized before the next liquidation cycle begins.

Vulnerability Details

In the `MarginLoanService` class, there's a method `getPeriod()` which defines how often the service runs:

```
@Override  
  
protected long getPeriod() throws Exception {  
    return globalConfig.getLong("marginloan.period.ms");  
}
```

Impact

In times of volatility, the liquidations might not be executed quickly enough to rescue positions from a bad debt, so it may cause insolvency of positions that have to be covered by the protocol.

Recommendation

Consider implementing a separate script/class that will run ad-hoc in order to increase liquidation speed in times of high volatility.

Note

The Ithaca team was apprised that they would introduce a more robust off-chain logic with increasing auction frequency to defend against increased market volatility.

Status

Resolved

[H-02] MarginLoanService - Exception handling in client processing could lead to complete failure of liquidation cycle

Description

The `MarginLoanService` processes multiple client accounts in a single cycle for liquidation and margin checks. However, the current implementation does not properly handle exceptions for individual clients, which could lead to the entire process failing if an exception is thrown for a single client. This is particularly critical in a liquidation context, where failing to process all accounts could lead to increased bad debt across the system.

Vulnerability Details

In the `MarginLoanService` class, the `doAction()` method processes multiple clients without proper exception handling:

```
// L112 onwards
```

```
protected void doAction() throws Exception {

    Map<Long, ClientMarginDbData> clientData = collectData(time);

    var updateDataFutures = buildClientDataUpdates(clientData);

    // ...

}
```

The `processClientData` method, called for each client, can throw exceptions:

```
//L352

private Pair<ClientMarginState, IthacaFeed.Client>
processClientData(ClientMarginDbData cd, Map<Currency, BigDecimal> usdcPrices) throws
Exception {

    switch (cd.getAccount().getAccountStatus()) {

        // ...

        default:

            throw new RuntimeException("Unexpected account status: " +
cd.getAccount().getAccountStatus());

    }

}
```

If an exception is thrown for any client, it will propagate up and cause the entire `doAction()` method to fail, preventing the processing of all subsequent clients.

Impact

If an exception occurs while processing any single client account, the entire liquidation cycle will fail.

This means that until the issue with the problematic account is resolved, no liquidations can take place for any accounts in the system.

volatile market conditions could lead to a rapid accumulation of debt across multiple accounts, potentially threatening the solvency of the entire protocol.

Recommendation

Create a separate queue or process for accounts that fail, allowing for manual review without impacting the main liquidation cycle. A single failure should not break the whole run.

Note

Due to Java Best Practises, the exceptions have to be handled in that exact way. However, the Ithaca team notified they will implement other off-chain solutions to mitigate this issue.

Status

Resolved

Medium

[M-01] MarginLoanService - Lack of emergency handling for failed liquidations and repayments

Description

The current implementation of the liquidation and repayment processes lacks robust error handling and emergency procedures. When liquidations or repayments fail, the system often simply logs the error and continues execution or returns null, without implementing any immediate corrective actions or escalation procedures.

Vulnerability Details

In the `repayLoans` method, failed repayments result in a null return, deferring the issue to a later time:

```
//RepayDetails repayLoans(...) line 690

    if (b.getAvailableAmount().compareTo(BigDecimal.ZERO) > 0) {

        if (!repayLoan(bcContracts, cd, b.getAvailableAmount(),
b.getCurrency(), b.getCurrency())) {

            //function failed. Try later

            return null;

        }

    }

[...]

//Line 723

    //repay

        if (!repayLoan(bcContracts, cd, b.getAvailableAmount(),
b.getCurrency(), c)) {

            //function failed. Try later
```

```
        return null;

    }
```

The `processLiquidation` method logs the liquidation result but doesn't have a mechanism to handle failed liquidations immediately:

```
//ClientData.java line 150

LiquidationHandler.LiquidationResult lr = processLiquidation(...);

[...]

LOGGER.warn("UNWIND process requested for client: " + clientId + ".
Liquidation result: " + lr);
```

In the `repayLoan` method, a failed transaction is logged but no immediate corrective action is taken:

```
//Line 769

if (!tr.isStatusOK()) {

    LOGGER.error("liquidateBorrowings has failed" + ...);

    return false;

}
```

Impact

Failed liquidations or repayments that are not immediately addressed can result in:

1. Undercollateralized positions remaining active for extended periods.
2. Increased losses due to delayed action in rapidly moving markets.

Recommendation

Implement an immediate retry mechanism with exponential backoff for failed liquidations and repayments. Consider adding an emergency escalation procedure for repeatedly failed liquidations, including manual intervention triggers.

Another approach is to implement partial liquidation/repayment functionality to ensure at least some risk is mitigated even if full liquidation fails.

Overall, there should be some possibility to perform a liquidation outside of the planned cycle.

Note

The Ithaca team will use an internal procedure to handle such issues.

Status

Resolved

[M-02] LiquidationHandler - Gradual price adjustments for liquidation orders may be too slow for volatile markets

Description

The `LiquidationHandler` uses a stepped approach to adjust prices for liquidation orders. This gradual adjustment (5% increments) may be too slow to respond effectively in highly volatile market conditions, potentially leading to increased losses or failed liquidations.

Vulnerability Details

In the `LiquidationHandler` class, the `getPriceMultiplier` method defines the price adjustment strategy:

```
public Pair<Account.AccountStatus, Double>
getPriceMultiplier(Account.AccountStatus accountStatus, boolean unwindAsk) {
    switch (accountStatus) {
```

```

// ...

case LIQUIDATION_UNWIND_REQUESTED:

case LIQUIDATION_REPAY_APPROVED:

case LIQUIDATION_STATUS_CHECK_REQUESTED:

case LIQUIDATION_REPAY_REQUESTED:

    //1st try

    if (unwindAsk) {

        return
Tuples.pair(Account.AccountStatus.LIQUIDATION_UNWIND_TRY_1_ASK, 1.05);

    } else {

        return
Tuples.pair(Account.AccountStatus.LIQUIDATION_UNWIND_TRY_1_BID, 0.95);

    }

case LIQUIDATION_UNWIND_TRY_1_ASK:

    if (unwindAsk) {

        //2nd try

        return
Tuples.pair(Account.AccountStatus.LIQUIDATION_UNWIND_TRY_2_ASK, 1.1);

    } else {

        //1st try for bid

        return
Tuples.pair(Account.AccountStatus.LIQUIDATION_UNWIND_TRY_1_BID, 0.95);

    }

    // ... more cases with similar gradual adjustments

}

}

```

This method adjusts the price in 5% increments (1.05, 1.10, 1.15 for asks; 0.95, 0.90, 0.85 for bids) with each liquidation attempt. In volatile markets, these adjustments may be too small and too slow to effectively liquidate positions.

Impact

In rapidly moving markets, gradual price adjustments may result in:

- Failed liquidations as market prices move faster than the adjustment rate.
- Increased losses to the protocol as positions become more underwater before successful liquidation.
- Accumulation of bad debt if positions can't be liquidated in time.

Recommendation

Consider implementing an emergency mode (like explained in H-01) where some positions can be liquidated right away or even rescued from accruing bad debt if LTV falls above 100% - in such case it should be possible to immediately sell the collateral at market price to minimise the further losses.

Note

The Ithaca team introduced changes to the off-chain protection mechanics which increases the liquidation intervals.

Status

Resolved

Low

[L-01] IthacaFeed.sol - Non-standard naming convention

Description

The contract uses naming for arguments in function initialize `accessController_`. While the usual approach for arguments naming convention would be `_accessController`.

Recommendation

Consider changing the variable name to `_accessController`.

Status

Acknowledged

[L-02] IthacaFeed.sol - Event should be more verbose

Description

The function `updateData` emits an event `emit ClientUpdated(backendId);`

However since events main purpose is to assure transparency to on-chain viewers, the event does not fully meet its role as it only informs which backend was the caller.

Recommendation

Include also `Client[] memory clients` in the event.

Status

Acknowledged

Centralisation

The Ithaca project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised



Conclusion

After Hashlocks analysis, the Ithaca project seems to have a sound and well-tested code base, however, now that our vulnerability findings have been acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd